

AI-Powered Smart City Insights Platform

Revised Project Plan — Team 11

Jia Lyu Jakub Jozef Dusza Jose Ricardo Arias Perez Mark Welf Atzberger

22.04.2026

1 Revised Direction and Motivation

The original proposal described an AI-powered smart city analytics platform ingesting Barcelona’s open data into a cloud data lake and enabling natural-language queries via Amazon Bedrock. This revised plan keeps that architecture intact and extends it with one additional interface layer: an **MCP server**.

1.1 What Changes and What Stays the Same

The AWS backend is unchanged: S3, Glue, Athena, DynamoDB, Lambda, API Gateway, Bedrock, QuickSight, CloudWatch. The data sources are unchanged: air quality, weather, noise, mobility. The addition is an MCP server that exposes the ingested data as standardised tools any AI assistant can call.

1.2 What is MCP and Why It Matters

MCP (Model Context Protocol) is an open standard originally developed by Anthropic and donated to the Linux Foundation in December 2025, backed by Google, Microsoft, AWS, OpenAI, and others. It defines how AI assistants communicate with external data sources and tools. Think of it as USB-C for AI: a universal connector so that any AI assistant can call any tool without custom integration code.

MCP has been adopted by Claude, Cursor, VS Code Copilot, Gemini, and Microsoft Copilot. From three implementations in October 2024 the ecosystem grew to over 16,000 indexed servers by November 2025. ChatGPT consumer support is in progress.

Why this matters for our project: existing MCP servers cover routing (Mapbox), geocoding (OpenStreetMap), and spatial analysis (CARTO), but none expose live environmental data for any European city. By building an MCP server on top of our data lake, we become the missing piece: any AI assistant can call our tools to answer questions like *”Is it safe to bike through Eixample today?”* or *”Why was the air quality bad near the port last Tuesday?”* — with real, live data, requiring no new app and no change in user behaviour.

1.3 Core Vision

Barcelona publishes excellent open data that is practically inaccessible to most citizens and tools. The goal of this project is to bring that data closer to actual people — not by building another

app that competes for attention, but by building infrastructure that plugs into the AI tools people already use every day.

When combined with existing routing MCPs (Mapbox, OSM), our server enables use cases neither could achieve alone: health-aware cycling routes that avoid high-pollution neighbourhoods, personalised morning briefings, anomaly explanations, Bicing availability integrated with air quality context. These capabilities improve passively over time as LLMs get better, as Barcelona adds sensors, and as MCP adoption widens — without changing our API.

2 Architecture Overview

The system has four layers. The original proposal covered layers 1,2 and 4; layer 3 (MCP server) is the addition.

1. **Data ingestion:** AWS Lambda functions poll each open data source on a schedule. Each function is independent so failures in one source do not affect others. Data writes to DynamoDB (latest readings, fast lookup) and S3 (historical, Parquet format).
2. **Storage and querying:** DynamoDB for real-time MCP queries; S3 + AWS Glue + Athena for historical SQL queries and trend analysis.
3. **MCP server (new):** A Python/Node.js server deployed on AWS Lambda via API Gateway, implementing the Streamable HTTP transport. Each tool reads from DynamoDB (real-time) or queries Athena (historical). Tool descriptions are engineered so AI assistants know when and how to combine them.
4. **Presentation:** Amazon Bedrock chatbot (built on top of the MCP server), Amazon QuickSight dashboard.

2.1 MCP Tools Exposed

conceptual, to be researched the actual useful exposed apis. these ones are the general, plausible ones:

- `get_air_quality(lat, lon, pollutant, timerange)`
- `get_weather(lat, lon)`
- `get_noise(area)`
- `get_bicing(lat, lon)` — live bike and dock availability
- `get_transit(area)` — bus/metro (optional, open task)
- `get_city_events(area, date)` — explains traffic/pollution anomalies

2.2 Demo

The centrepiece demo is a web chatbot that has simultaneous access to our MCP server and an existing routing MCP (Mapbox or OSM). The user asks a natural-language question; the AI calls whichever tools are relevant, combines the results, and returns an explanation plus a visual map or chart. Example:

"Give me a bike route from Gràcia to Barceloneta that avoids high pollution right now."
→ assistant calls `get_air_quality` (NO2 elevated in Eixample), calls Mapbox routing avoiding Eixample, calls `get_bicing` near origin, returns route on map with explanation.

3 Work Structure: Vertical vs Horizontal

3.1 Horizontal Tasks (Everyone)

These are not owned by one person. Every team member is responsible for completing them.

- **Learn MCP basics:** read enough to understand what an MCP tool is, how AI assistants call tools, and what a well-designed tool description looks like. This gives the right intuition for intermediate decisions throughout the project.

Suggested resource: <https://modelcontextprotocol.io/docs>. Estimated time: 2–3 hours.

- **Learn the project data flow:** understand how data moves from source → Lambda → DynamoDB/S3 → MCP tool. You need this to implement your vertical correctly.
- **Final report and presentation:** everyone contributes to their section.

3.2 Vertical Tasks (Owned by One Person, End to End)

Each vertical covers: research the data source, build the Lambda ingestion function, define the DynamoDB schema for your data, implement the MCP tool function for your area. You own that domain from raw data to queryable tool.

The principle is: *whoever owns the data source owns the MCP tool for it*. This keeps knowledge concentrated and avoids handoff overhead.

4 Data Sources per Vertical

Each person should research their assigned area with two questions in mind: (1) what data is available and how does it update? (2) how do we make this data maximally useful for an end user or AI assistant?

Area	Primary Sources	Key Fields
Air quality	XVPCA via Open Data BCN	NO2, PM10, O3, PM2.5; 12 stations; hourly
Weather	Meteocat API; Open-Meteo (backup)	Temp, wind, humidity, UV, precipitation; hourly
Noise	Open Data BCN noise network	dB levels by area; strategic noise maps
Mobility — bikes	BSM Bicing GBFS	Bikes + docks available per station; every 5 min
Mobility — transit (open)	TMB API (bus/metro)	Lines, occupancy, delays; real-time
City events (open)	Barcelona events calendar	Events, location, date; explains anomalies

Note on Data Freshness and Reliability

All Lambda ingestion functions should write the reading timestamp alongside the data value. The MCP tool must return this timestamp so the AI assistant can communicate data age to the user. If a source API is unavailable, return the last cached DynamoDB value with its age rather than an error.

5 Task Definition and Time Management

Total Deadline: 29 May

Total Work Hours: 150h (160h minus 10h already spent on proposals)

Estimated per person: 37.5h

Tasks are labelled **H** (horizontal — everyone) or **V** (vertical — one owner). Priority is marked **HARD** (must be done, blocks others or the demo) or **ADDITIONAL** (valuable but droppable if time is short).

#	Task	Est.	H/V	Priority	Description
0.1	Management	10h	V	Regular	Deadlines, communication, keeping scope on track. Ongoing.
1	MCP onboarding	2–3h	H	Regular	Everyone reads MCP docs and understands: what a tool is, what a tool description looks like, how an AI assistant calls tools. No deep implementation knowledge required — just enough context to work well and make good decisions throughout.

#	Task	Est.	H/V	Priority	Description
2	AWS setup & IAM	5h	V (1 person)	HIGH	Create shared AWS account. Set up one IAM role per team member with least-privilege access. Create shared S3 bucket and DynamoDB table structure that everyone writes to. Document clearly: how to create a Lambda, what table schemas exist, how to run a test write. Deliverable: every other team member can deploy a Lambda and write to shared DynamoDB by end of this task. This person should start immediately and finish this week so others are not blocked.
3a	Air quality vertical	10h	V (Mark)	Regular	Research XVPCA / Open Data BCN API. Build Lambda ingestion (hourly poll). Define and populate DynamoDB schema. Implement <code>get_air_quality()</code> MCP tool function.
3b	Weather vertical	10h	V (Jia)	Regular	Research Meteocat API (free key required) and Open-Meteo as backup. Build Lambda ingestion (hourly). DynamoDB schema. Implement <code>get_weather()</code> MCP tool function.
3c	Noise vertical	10h	V (Kuba)	Regular	Research Open Data BCN noise monitoring network. Note: noise data is less real-time than other sources — assess what update frequency is realistic and design accordingly. DynamoDB/S3 schema. Implement <code>get_noise()</code> MCP tool function.
3d	Mobility bikes vertical	— 10h	V (Jose)	Regular	Research BSM Bicing GBFS endpoint (standard JSON, no key, updates every 5 min). Build Lambda ingestion. DynamoDB schema. Implement <code>get_bicing()</code> MCP tool function.

#	Task	Est.	H/V	Priority	Description
3e	Mobility — transit vertical	8h	V (open)	Regular	Research TMB API for Barcelona bus and metro. Build Lambda ingestion. DynamoDB schema. Implement <code>get_transit()</code> MCP tool function. Open task — anyone can pick this up. Fine to drop if nobody takes it.
3f	City events ver- tical	5h	V (open)	additional	Research Barcelona events calendar API. Daily Lambda poll. DynamoDB schema. Implement <code>get_city_events()</code> MCP tool function. Adds contextual explanations (e.g. concert at Palau Sant Jordi → elevated traffic). Open task.
4	S3 / Glue / Athena	10h	V (1 per- son)	Regular	Set up S3 data lake with Parquet storage for historical data from all verticals. Configure AWS Glue crawlers and catalog. Validate Athena SQL queries over historical data. This enables trend queries and anomaly detection.
5	MCP server	15h	V (open, Mark/Jose from notes)	Regular	Build the MCP server itself: Python or Node.js, deployed on Lambda via API Gateway, Streamable HTTP transport. Integrates all tool functions from each vertical into one server. Writes the tool descriptions carefully (critical: poor descriptions = poor AI orchestration). Rate limiting and basic error handling from day one.

#	Task	Est.	H/V	Priority	Description
6	Demo chatbot & visualiser	15h	V (open, Mark/Jose from notes)	Later	Build the web demo: React frontend with embedded map (Mapbox GL JS), chatbot UI powered by Claude API (bedrock) using our MCP server + Mapbox MCP simultaneously. Chatbot responses can control the map (render routes, show pollution overlay). Implement the health-aware routing demo query end to end.
7	QuickSight dashboard	10h	V (1 person)	additional	Create QuickSight dashboard showing key metrics from the data lake: air quality trends, traffic patterns, Bicing availability heatmap. Kept as additional since the MCP demo is the primary deliverable.
8	Telegram bot	5h	V (Jose from notes)	additional	Simple Telegram bot: <code>/air Eixample</code> , <code>/bicing Gracia</code> . Calls the MCP tools directly. Very low effort once MCP server exists. Open task.
10	Testing	10h	H	Later	End-to-end testing of data pipeline, MCP tool responses, and demo chatbot. Test with varied query types. Validate data freshness and error fallback. Deadline: 22.05.
11	Presentation	10h	H	Later	Prepare demo script and slides. Each person presents their vertical. Deadline: 29.05.
12	Report writing	10h	H	Later	Final report. Each person writes their section. Deadline: 29.05.

6 Immediate Next Steps This Week

The following must happen before anything else is unblocked.

1. **Someone takes AWS setup (Task 2)** and starts today. Deliverable by end of this week: shared AWS account, IAM roles for each person, shared DynamoDB table structure, documented instructions so everyone else can connect their Lambda.

2. **Everyone completes MCP onboarding (Task 1)** independently.

Read: <https://modelcontextprotocol.io/docs>. Focus on: what is a tool, what does a tool description look like, how does an AI assistant decide which tool to call. 2–3 hours. (watch some videos, etc. whatever is easiest for you)

3. **Each person begins data source research for their vertical** (Tasks 3a–3d): find the API, download sample data, understand update frequency and data format, identify what fields are most useful for an AI assistant or end user. This research feeds into the Lambda design.
4. **Open tasks are claimed or dropped**

7 Timeline

Deadline	Milestone
This week (25.04)	AWS setup complete. Everyone has IAM access. MCP onboarding done. Data source research started for all four verticals.
01.05	All four core Lambda ingestion functions deployed and writing to DynamoDB. Data flowing for air quality, weather, noise, bikes.
08.05	All MCP tool functions implemented. MCP server running (at least locally or on Lambda). S3/Glue/Athena configured. Athena queries validated.
15.05	Demo chatbot functional end to end. Health-aware routing demo working. Bedrock integration done. QuickSight dashboard (if taken).
22.05	Testing complete. All known issues resolved. Report sections drafted.
29.05	Final presentation. Report submitted.

8 Workload Estimate

Task	Hours	Notes
MCP onboarding (H)	2–3h	Everyone
AWS setup & IAM	5h	1 person
Data vertical (each)	10h	4 people $\times$ 10h
S3 / Glue / Athena	10h	1 person
MCP server	15h	1–2 people
Demo chatbot	15h	1–2 people
QuickSight (optional)	10h	1 person
Telegram bot (optional)	5h	1 person
Testing (H)	10h	Everyone
Presentation (H)	10h	Everyone
Report (H)	10h	Everyone
Management	10h	1 person
Core total	$\sim$150h	Excluding optional tasks

The core workload (excluding optional tasks) is approximately 150 hours across four people, or 37.5 hours per person. Open tasks (transit, events, Telegram, QuickSight) add up to $\sim$ 28 hours and can be distributed based on who picks them up.